

Testing Bearer Tokens

Check login, tokens, and logout with `curl` and in the browser.

Learning Goals

- Test four cases for every protected route: no token, invalid token, valid token, and logged out.
- Read the status codes 200, 201, and 401 and explain what each means.
- Explain why tokens are stored as hashes and can be revoked.

Start the server that `run1-5.sh` built:

```
cd student-api  
php artisan serve --port=8080  
# WSL2: php artisan serve --host=0.0.0.0 --port=8080
```

- All `curl` commands below are also in `test1-5.sh`.
- There are four cases to test for every protected route: **no token**, **invalid token**, **valid token**, and **logged out**.

Test: Public Route

```
php artisan route:list --path=api  
curl -H 'Accept: application/json' http://localhost:8080/api/students/1
```

- Reading students works without a token.
- Always send `Accept: application/json`. Without it, Laravel may try to redirect an unauthenticated request to a login page instead of returning JSON.

Test: Log In

```
curl -X POST http://localhost:8080/api/login \  
  -H 'Accept: application/json' -H 'Content-Type: application/json' \  
  -d '{"email":"student@university.edu","password":"student123"}'
```

```
{"message":"Login successful","token":"1|Xk3v9...",  
  "token_type":"Bearer","expires_in":3600,"user":"Student"}
```

A wrong password returns **401** .

Save the token in a variable, so you do not copy it by hand:

```
TOKEN="1|Xk3v9..." # paste the token from the login response
```

Test: Missing, Invalid, and Valid Token

```
curl -i -H 'Accept: application/json' \  
http://localhost:8080/api/me
```

```
curl -i -H 'Accept: application/json' \  
-H 'Authorization: Bearer invalid-token' \  
http://localhost:8080/api/me
```

```
curl -i -H 'Accept: application/json' \  
-H "Authorization: Bearer $TOKEN" \  
http://localhost:8080/api/me
```

- The first two return **401**. The last one returns **200** and the user.

Test: Protected Write

```
curl -i -X POST http://localhost:8080/api/students \  
  -H 'Accept: application/json' -H 'Content-Type: application/json' \  
  -d '{"name":"Grace Lee","email":"grace.lee@university.edu","age":20,"major":"Computer Science","year":2}'
```

- Without a token: **401**. Add `-H "Authorization: Bearer $TOKEN"` and the same request returns **201**.
- In Lesson 4, this request worked without any token. Now it does not.

Look Inside the Token Table

```
mysql -u ase230 -p studentdb -e \  
"SELECT id, tokenable_id, name, LEFT(token,12) AS token_hash FROM personal_access_tokens"
```

id	tokenable_id	name	token_hash
1	1	api-token	8c1f4e9a02b7

- The `token` column holds a hash. It does **not** match the string you sent.
- Each login creates a new row.

Log Out

```
curl -X POST http://localhost:8080/api/logout \
  -H 'Accept: application/json' -H "Authorization: Bearer $TOKEN"

curl -i -H 'Accept: application/json' \
  -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/me
```

- The second request returns **401**: the token row was deleted, so the same string is no longer valid.
- This is the big advantage over the Module 1 demo, where tokens were fixed strings that could never be revoked.

Test from the Browser: run1-5-tests

Typing `curl` commands is slow. Open a web page that runs the tests for you.

```
open run1-5-tests/index.html          # macOS  
# WSL2: open the file from Windows Explorer
```

- It is plain HTML and JavaScript, and it calls the API with `fetch()`. The file is in `run1-5-tests/`.
- Enter the email and password at the top. The default is the `student demo` user.
- This is the Bearer version of the `run1-4-tests` page in Lesson 4 (`4-3`).

What the Tests Check

Part	Test	What it checks
1	Connection, wrong password, login	200 , then 401 , then a token that looks like id secret
2	GET /api/me	no token: 401 ; invalid token: 401 ; valid token: 200 and the user
3	Students	reads are public; create without a token is 401 , with a token is 201
3	Update, delete, validation	200 , then 404 after delete; bad input gives 422
		200 , then the user is i

How the Page Sends a Token

```
async function request(path, method = 'GET', data = null, bearer = null) {  
  const headers = { 'Accept': 'application/json' };  
  if (data !== null) headers['Content-Type'] = 'application/json';  
  if (bearer) headers['Authorization'] = `Bearer ${bearer}`;  
  const response = await fetch(baseUrl() + path, { method, headers, ... });  
  ...  
}
```

- `bearer` is the token, or `null` for no `Authorization` header. The same function tests all four cases.
- The token is kept in a JavaScript variable, not in `localStorage`. Reloading the page logs you out.
- The browser can call the API from another origin because Laravel sends `Access-Control-Allow-Origin` for `/api/*` (CORS).

Security Checklist

Rule	How this lesson handles it
Hash passwords	<code>'password' => 'hashed'</code> cast in <code>User</code>
Hash stored tokens	Sanctum stores only the SHA-256 hash
Expire tokens	<code>createToken(..., now() ->addHour())</code>
Revoke tokens	<code>currentAccessToken()->delete()</code>
Use HTTPS	Required in production; the local demo uses HTTP
Limit login attempts	Next step: the <code>throttle</code> middleware

Apply This to Your Project

- Add `HasApiTokens` to your `User` model and run the Sanctum migration.
- Write a `login` action that returns a token, and a `logout` action that revokes it.
- Decide which routes are public and which need `auth:sanctum`.
- Test all four cases for each protected route: no token, invalid token, valid token, and logged out.
- Compare your results with `test1-5.sh` and the `run1-5-tests` page.

Next: Optional HTML

Lessons 1 to 5 build a complete API with a database and Bearer tokens.

- **Lesson 6 (optional)** adds HTML pages to the same student data. You can skip it.